VECTOR

MICROSAR Ethernet Driver

Technical Reference

Infineon AURIX TriCore Tc3xx Ethernet Controller IP

Version 6.1.0

Authors	Benjamin Groebner, Abhishek Halivan, Kevin Gomes, Christian Nagengast, Tobias Schwindl
Status	Released

Document Information

History

Author	Date	Version	Remarks
Benjamin Groebner	2017-08-24	0.1.0	Creation of document
Benjamin Groebner	2018-01-04	0.2.0	TASK-60515 - Provide DrvETh_Tc3xx Beta version using TriBoard TC3X9 V2.0
Benjamin Groebner	2018-02-26	1.0.0	TASK-65966 - Review Integration and Core Update
Benjamin Groebner	2018-05-23	2.0.0	FEAT-3644 - Support QoS/FQTSS by DrvEth_Tc3xxEthAsr
Benjamin Groebner	2019-03-20	3.0.0	▶ ESCAN00101437 Possible memory violation in case of multi Eth driver configuration ▶ ESCAN00101808 Compiler warning: Calling function without prototype Appl_UnlockEndinit / Appl_LockEndinit ▶ ESCAN00101709 Missing overflow handling during global time retrieval
Benjamin Groebner	2019-04-04	3.0.1	▶ ESCAN00102625 Possible loss of interrupt could lead to stuck communication on Ethernet ▶ ESCAN00101663 Memory corruption possible if RX buffers are configured smaller than 1522 bytes
Abhishek Halivan	2019-08-27	4.0.0	ETH-786 Update DrvEth_Tc3xxEthAsr to MSR4-R23 DrvEth__coreAsr
Abhishek Halivan	2019-09-27	4.0.1	▶ ESCAN00103309 DMA Bus Mode register field INTM is initialized with reserved value ▶ ESCAN00103167 Selecting Ethernet port speed to 1000Mbit/s sets speed to 2500Mbit/s
Kevin Gomes	2020-02-24	4.0.2	▶ ESCAN00104650 TCP/UDP/ICMP checksum offloading not working for TX frames with more than 964 bytes
Abhishek Halivan	2020-05-12	4.1.0	▶ Allow triggering PPS signal from Tc3xx to external chip on a given timer value in the MAC
Christian Nagengast	2020-06-26	5.0.0	▶ Migrate DrvEth_Tc3xxEthAsr to DrvEth__coreAsr 9.1.1
Tobias Schwindl	2020-10-07	6.0.0	▶ Introduce Errata sheets as reference
Christian Nagengast	2020-11-06	6.1.0	▶ ETH-3295 Update QoS feature description

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	Specification of Ethernet Driver	R4.1.1
[2]	AUTOSAR	Specification of Development Error Tracer	R4.1.1
[3]	AUTOSAR	Specification of Diagnostic Event Manager	R4.1.1
[4]	AUTOSAR	General Specification of Basic Software Modules	RV1.0.0
[5]	Vector	TechnicalReference Ethernet Driver	see delivery
[6]	Infineon	AURIX TC3xx User's Manual Part 1+2	1.0.0 – 1.4.0
[7]	Infineon	TC38x User's Manual Appendix	2.3.0
[8]	Vector	TriCore ENDINIT Protection Handling	see delivery
[9]	Infineon	Errata Sheet TC38x AD	1.5
[10]	Infineon	AURIX TC3xx Safety Manual	1.10

Scope of the Document

This technical reference describes the specific use of the specific controller driver software. It supplements the general Ethernet driver technical reference [5].



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.



Note
This document describes aspects that apply for Ethernet Driver modules provided by Vector, which are only used for Infineon Tc3xx Ethernet modules. For this reason, the corresponding infixes of APIs, macros, parameters etc. are replaced by Tc3xx or, if uppercase is used, TC3XX.

The platform independent aspects of the Ethernet Driver are discussed in [5], where the above-mentioned infix is replaced by the markers <Driver> and <DRIVER>.

Contents

1 Introduction 7

2 Hardware Overview 8

3 Functional Description 9
3.1 Features 9
3.1.1 Deviations 9
3.1.2 Additions/ Extensions 9
3.1.3 Limitations 9
3.1.3.1 Checksum Offloading Limitations 9
3.2 Initialization 9
3.3 States 9
3.4 Main Functions 10
3.5 Error Handling 10
3.5.1 Development Error Reporting 10
3.5.2 Production Code Error Reporting 10
3.6 Checksum Offloading 10
3.7 Quality of Service (QoS) 10
3.7.1 Queue Priorities 11
3.7.2 VLAN Priority Mapping 11
3.8 FQTSS (Traffic Shaping) 11
3.9 PPS (Pulse Per Second) Output Signal 12

4 Integration 14
4.1 Embedded Implementation 14
4.2 Exclusive Areas 14
4.3 Memory Mapping 14
4.4 Interrupts 14
4.4.1 Interrupt Mapping 14
4.4.2 Interrupt Prioritization 15
4.5 Operational Preconditions 15
4.5.1 ETH_GPCTL register and port configuration 16

5 API Description 17
5.1 Structure Definitions 17
5.1.1 Eth_TimeStampType 17
5.2 API Table 17
5.2.1.1 Eth_30_Tc3xx_ArmPpsEventOnTime 17

6 Configuration 18

7 Glossary and Abbreviations 19
7.1 Glossary 19
7.2 Abbreviations 19

8 Contact 20

Illustrations

Figure 3-1 Example configuration for Traffic Shaping 12

Tables

Table 2-1 Hardware Overview 8
Table 3-1 Vector Driver Feature Set Support 9
Table 3-2 Checksum Offloading Capability 10
Table 3-3 TX queue to ring priority mapping 11
Table 3-4 Rx queue to ring priority mapping 11
Table 3-5 Hardware support for Traffic Shaping 12
Table 4-1 Memory Mapping 14
Table 4-2 Interrupt Mapping 15
Table 4-3 Pin configuration TriBoard for Tc389 16
Table 5-1 Eth_30_Tc3xx_ArmPpsEventOnTime 17
Table 7-1 Glossary 19
Table 7-2 Abbreviations 19

1 Introduction

This document describes the specific functionality of the driver either limiting or extending the function set described in [5].

In addition to that specific integration information for the driver itself and specific supported derivatives and use-cases beyond the general ones described in [5] are given.

2 Hardware Overview

The following table summarizes the supported hardware platforms and settings this Ethernet Driver can be used on. It also provides information on the documentation used during the development of the driver for detailed reference to hardware specific information.

Derivatives	Infineon Aurix Tc3xx Derivates with Gigabit Ethernet module
Compiler	Tasking
Hardware manuals	See [6], [7]
Errata sheets	[9] GETH_TC.002 and GETH_AI.H001
Safety manuals	[10]
Specifics	-
Used registers (offset to base register offset)	MAC: 0x0000 - 0x00B8 and 0x0B00 – 0x0B1C0 MMC: 0x0700 - 0x07D4 MTL: 0x0C00 - 0x0DF0 DMA: 0x1000 - 0x2040
Operating modes	-
Hardware features related to independence or partitioning	none
Access mechanism	Memory mapped registers
Hardware diagnostics	none

Table 2-1 Hardware Overview

3 Functional Description

3.1 Features

The features listed in the following tables cover the specific functionality specified for the Eth differing from the one defined in [5].

The following table gives a short overview over the functionality provided by the driver regarding [5]. For deviations and limitation see the following subsections.

Feature	Driver/Hardware support - = not available ■ = available on specific derivatives ■ = available
Time Synchronization	■
Quality of Service	■
FQTSS (Traffic Shaping)	■
Checksum Offloading	■
Receive Buffer Segmentation	- (not supported by this driver)
Runtime environment INTEGRITY	- (not supported by this driver)

Table 3-1 Vector Driver Feature Set Support

3.1.1 Deviations

There are no deviations.

3.1.2 Additions/ Extensions

There are no additions or extensions.

3.1.3 Limitations

3.1.3.1 Checksum Offloading Limitations

The checksum offloading feature is supported with limitations. Refer to section 3.6 for more details.

3.2 Initialization

In contrast to [5] the driver needs following specific initialization procedure:

During the execution of Eth_30_Tc3xx_Init() the Ethernet driver needs to configure the Ethernet clock register (ETH_CLC).

To be able to perform the configuration, the user must guarantee full write access to this register, which is protected by the watchdog-control-register. The user has to provide two functions to disable (Appl_UnlockEndinit()) and afterwards enable (Appl_LockEndinit()) the write protection by accessing the watchdog register. Please refer to Application Note [8] for details.

3.3 States

The driver doesn't provide additional states to the ones listed in [5]

3.4 Main Functions

The driver doesn't provide additional main functions.

3.5 Error Handling

3.5.1 Development Error Reporting

The driver doesn't provide additional development error reporting functionality.

3.5.2 Production Code Error Reporting

The driver doesn't provide additional production error reporting functionality.

3.6 Checksum Offloading

The driver/hardware can offload the checksum calculation of following protocols:

Protocol	Driver/Hardware support - = not available ■ = available
ICMPv4	■
IPv4	■
UDP over IPv4	■
TCP over IPv4	■
ICMPv6	■
IPv6 with Routing Header Option	-
IPv6 with Destination Header Option	■
IPv6 with Hop by Hop Header Option	■
UDP over IPv6	■
TCP over IPv6	■

Table 3-2 Checksum Offloading Capability

3.7 Quality of Service (QoS)

Quality of Service is supported for transmission and reception of frames. For frame transmission, up to four queues provided by the hardware can be used. When a frame is sent, it is sorted into the corresponding transmission queue based on the VLAN-priority, before a DMA-request is triggered. When a frame is received, its VLAN-priority is analyzed by a separation filter in hardware and sorted into the corresponding reception queue. Queues with a higher priority are processed before queues with lower priority.



Note
Due to hardware limitations only 2 Tx Queues can be configured when Checksum Offloading is enabled



Note
Quality of Service can be applied on VLAN tagged traffic only because it relies on the priority encoded in the VLAN TCI (Tag Control Information).

3.7.1 Queue Priorities

The supported queue priorities can be identified in the configuration tool by the parameter `Priority` (see Table 3-3) and are assigned to the transmission queues as follows:

Queue	Priority
Q0	0
Q1	1
Q2	2
Q3	3

Table 3-3 TX queue to ring priority mapping

For receiving frames, the following priorities are assigned to the available reception queues:

Queue	Priority
Q0	0
Q1	1
Q2	2
Q3	3

Table 3-4 Rx queue to ring priority mapping

3.7.2 VLAN Priority Mapping

The driver provides the possibility to map VLAN-priorities by configuration to the transmission- and reception queues described in section 3.7.1. This means that for example after mapping VLAN-priority 7 to reception queue 3, all frames holding this VLAN-priority will be received with the highest priority. There is no further restriction to the mapping besides that each VLAN-Priority (0-7) must be assigned and that every queue except the queue with priority 0 needs an assigned VLAN-Priority.

A detailed description of how to configure the queues is not necessary because DaVinci Configurator completely guides through the setup.

3.8 FQTSS (Traffic Shaping)

The TriCore GETH contains three Traffic Shaping units. Table 3-5 shows which queues are available for traffic shaping.

Queue	Traffic Shaping Availability - = not available ■ =available
Q0 (Best effort)	-
Q1 (Network Control)	■
Q2 (Class B)	■
Q3 (Class A)	■

Table 3-5 Hardware support for Traffic Shaping

Figure 3-1 shows an example configuration, where a Bandwidth Limit of 12 000 000 Bit/s is configured for the third transmission queue. This is 1.2% of the Port Transmit Rate of 1000Mbit/s, which is configured in the transceiver settings. The type of the hardware queue with priority 3 is Traffic Class A. This information is hardware dependent and is set by DaVinci Configurator automatically. In this example VLAN priority 6 and 7 are mapped to the queue with priority 3. Thereby this traffic shaper is active for all traffic sent using VLAN priority 6 or 7, which are handled in queue 3.

Protocols like SRP (Stream Reservation Protocol) in an AVB context need this feature to manage the reserved streams. For AVB conformance validation 4 queues are required. As a consequence, Checksum Offloading is not supported together with AVB conformance validation.

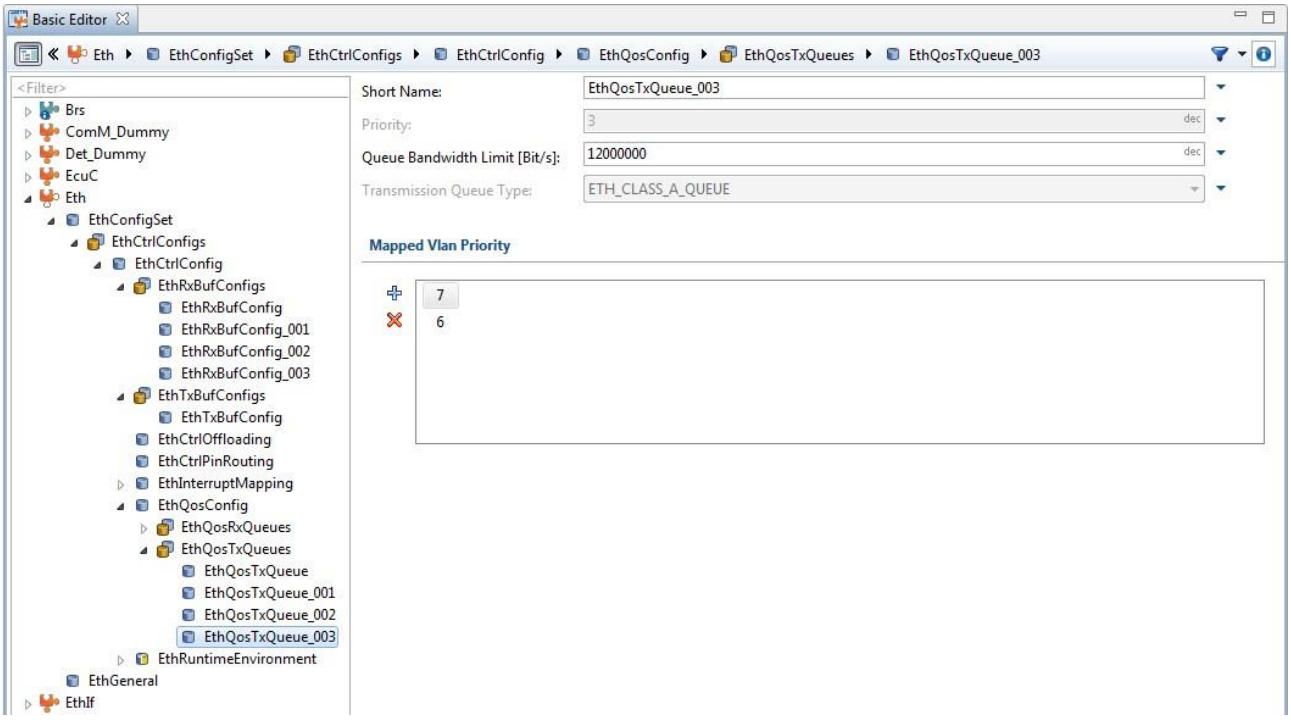


Figure 3-1 Example configuration for Traffic Shaping

3.9 PPS (Pulse Per Second) Output Signal

In some time synchronization scenarios, e.g. if different clock sources are present, it is necessary to provide additional hardware synchronization between participants. GETH

provides the PPS signal output based on the given Timestamp to achieve the synchronization.

The API Eth_30_Tc3xx_ArmPpsEventOnTime provides enabling single pulse generation during runtime, which can be used to achieve time synchronization on an external device. The configured start time value in the PPS time registers Target_Time_Seconds and Target_Time_Nanoseconds (value range not exceeding 31bit) are compared with PTP time register by the GETH. If the time register values are equal, a single pulse will be generated on the respective GETH_PPS port pin. The width and interval of the signal pulse is calculated depending on the PTP clock frequency provided in the GETH. The parameter Eth_Tc3xx/Eth/EthConfigSet/EthCtrlConfig/EthPtpRefClock configures the frequency. If there is no value configured, the default frequency 50MHz is used. Refer to 5.2.1.1 for API details of Eth_30_Tc3xx_ArmPpsEventOnTime.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR Eth into an application environment of an ECU.

4.1 Embedded Implementation

The delivery of the Eth in any case contains the files which are described in see [5].

This driver doesn't provide additional files to the ones listed in [5].

4.2 Exclusive Areas

This driver doesn't provide additional exclusive areas.

4.3 Memory Mapping

This section describes the specific memory mapping that must be applied for the buffers and descriptors of the driver.

Derivative	Section	Alignment	Memory
TC3xx	Buffer-Section	16 Byte	Scratch-Pad-RAM
	Descriptor-Section	16 Byte	System-RAM

Table 4-1 Memory Mapping



Caution
Previous table can't be assumed to be applicable for any derivative supported by the driver.

Please see the derivatives respective reference manual to ensure that no specific restrictions apply for it.

4.4 Interrupts

4.4.1 Interrupt Mapping

This section describes the mapping between the interrupt events given by the Ethernet Controller and used by the driver and their respective mapping to interrupt sources of the Interrupt Controller integrated on the derivative.

Derivative	Interrupt Source	Interrupt Event
TC3xx	Ethernet Interrupt Queue 0 Receive	Ethernet RX on Queue 0
	Ethernet Interrupt Queue 0 Transmit	Ethernet TX on Queue 0
	Ethernet Interrupt Queue 1 Receive	Ethernet RX on Queue 1
	Ethernet Interrupt Queue 1 Transmit	Ethernet TX on Queue 1
	Ethernet Interrupt Queue 2 Receive	Ethernet RX on Queue 2
	Ethernet Interrupt Queue 2 Transmit	Ethernet TX on Queue 2
	Ethernet Interrupt Queue 3 Receive	Ethernet RX on Queue 3
	Ethernet Interrupt Queue 3 Transmit	Ethernet TX on Queue 3

Table 4-2 Interrupt Mapping



Note
Previous table doesn't contain all supported derivatives.

If the used derivative isn't listed in the table, please see the respective reference manual of the derivative to retrieve an applicable mapping.

4.4.2 Interrupt Prioritization

The driver doesn't require specific prioritization of interrupts.



Caution
Regarding the upper layers of the MICROSAR Stack (TcpIp, SoAd) it must be ensured that the Receive -Interrupts of the available QoS queues are all mapped to the same priority, so they aren't able to interrupt each other. (Same requirement applies to the Transmit-Interrupts)



Caution
Previous table can't be assumed to be applicable for any derivative supported by the driver.

Please see the derivatives respective reference manual to ensure that no specific restrictions apply for it.

4.5 Operational Preconditions

This section describes the preconditions that must be fulfilled for either a specific use-case or a specific derivative for the driver to be operational after module initialization.

4.5.1 ETH_GPCTL register and port configuration

The TriCore Ethernet MAC provides different general purpose I/O lines that can be chosen alternatively. Depending on the I/O line and the kind of interface (RGMII, RMII or MII), the ports and the ETH_GPCTL register must be adapted by the user.

This can be done in the DaVinci Configurator Pro.

For Infineon Triboards with TC389 the following settings need to be applied:

Parameter	Value
GPCTLALT0	ETH_ALT_2
GPCTLALT1	ETH_ALT_0
GPCTLALT2	ETH_ALT_0
GPCTLALT3	ETH_ALT_0
GPCTLALT4	ETH_ALT_0
GPCTLALT5	ETH_ALT_0
GPCTLALT6	ETH_ALT_0
GPCTLALT7	ETH_ALT_0
GPCTLALT8	ETH_ALT_0
GPCTLALT9	ETH_ALT_0
GPCTLALT10	ETH_ALT_0

Table 4-3 Pin configuration TriBoard for Tc389

5 API Description

Please refer [5] for more information about the API description.

5.1 Structure Definitions

5.1.1 Eth_TimeStampType

Please refer [5] for description of Eth_TimeStampType.

5.2 API Table

5.2.1.1 Eth_30_Tc3xx_ArmPpsEventOnTime

Prototype	
`Std_ReturnType` **`Eth_30_Tc3xx_ArmPpsEventOnTime`** `(uint8 CtrlIdx, Eth_TimeStampType *TimestampPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
TimestampPtr [in]	Provided time stamp
Return code	
Std_ReturnType	E_OK - PPS signal trigger configuration was successful / is not enabled
	E_NOT_OK - PPS signal trigger configuration failed
Functional Description	
Configures the controller to trigger the PPS single pulse on the given time.	
Particularities and Limitations	
- Module is initialized - The given time stamp of type Eth_TimeStampType must not use a nanoseconds value exceeding a numerical value greater than 31 bit. - If the API is not enabled, it returns E_OK	
Call context	
> TASK > This function is synchronous > This function is reentrant	

Table 5-1 Eth_30_Tc3xx_ArmPpsEventOnTime



Caution

The given time stamp of type Eth_TimeStampType must not use a nanoseconds value exceeding a numerical value greater than 31 bit. Please refer [5] for description of Eth_TimeStampType.

6 Configuration

The driver doesn’t need specific configuration.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
DaVinci Configurator PRO	Generation tool for MICROSAR basis software

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
AVB	Audio Video Bridging
BSW	Basis Software
CAD	Component Abstract Design
ECU	Electronic Control Unit
FQTSS	Forwarding and Queuing for Time-Sensitive Streams (IEEE 802.1Qav)
ICMPv4	Internet Control Message Protocol over IPv4
ICMPv6	Internet Control Message Protocol over IPv6
IPv4	Internet Protocol Version 4
IPv6	Internet Protocol Version 6
I/O	Input / Output
MAC	Media Access Control
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
MII	Media Independent Interface
PPS	Pulse Per Second
PTP	Precision Time Protocol
QM	Quality Management
QoS	Quality of Service
RAM	Random Access Memory
RGMII	Reduced Gigabit Media Independent Interface
RMII	Reduced Media Independent Interface
RX	Receive
SRP	Stream Reservation Protocol
TCP	Transmission Control Protocol
TX	Transmit
UDP	User Datagram Protocol
VLAN	Virtual Local Area Network

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com